

Prueba Técnica

Desarrollador/a Full-stack Semi-Senior

1. Contexto

Trabajas en **Northwind**, una empresa que vende software de gestión a otras empresas (SaaS B2B). Tienen alrededor de **420 clientes activos** y facturan mensualmente unos **USD 380.000**, con tickets que van desde USD 200 hasta USD 15.000.

El equipo de finanzas son **2 personas** y gestionan la cobranza con una planilla de Google Sheets que actualizan a mano los lunes. A partir de ahí deciden a quién llamar, a quién mandar un recordatorio, y a quién dejar tranquilo.

Los problemas vienen mezclados:

- **La mora subió del 6% al 14%** en un año y nadie sabe bien por qué.
- **No todos los morosos son iguales:** hay clientes grandes que pagan a 75 días por proceso interno, startups que se quedan sin caja de un día para otro, y clientes "zombi" que siguen consumiendo el servicio sin pagar hace 90 días.
- **Los recordatorios automáticos actuales son ruido:** los clientes grandes los ignoran, los chicos se ofenden, y los que están en problemas reales reciben los mismos 3 emails que todos.

Lo que la CEO te dijo, textualmente:

"Necesitamos una herramienta para gestionar la cobranza y anticiparnos a los problemas. Algo que nos ayude a saber dónde poner foco. Tienes 3 días. Sé que mi requerimiento es poco, pero confío en tu criterio. Cualquier duda me preguntas, pero adelántate todo lo que puedas."

Eso es todo. No hay documentos de requerimientos, no hay diseño, no hay product manager intermediario. Sabes que la CEO responde lento (4-6 horas) y que puedes pedir 30 minutos con la analista de cobranza, que es quien más sabe del problema real. No tienes acceso a datos reales por privacidad, pero puedes asumir cualquier estructura razonable y poblarla con datos sintéticos.

2. Qué esperamos que entregues

Una solución end-to-end que tú definas, que incluya:

- Un análisis del problema y de las decisiones de producto que tomaste (qué incluiste, qué descartaste, por qué).
- Separación del requerimiento principal en sub-requerimientos.
- Un backend funcional con persistencia en base de datos relacional (Go preferido; NestJS válido).
- Un frontend en React que consuma ese backend y exponga la funcionalidad principal.
- Un README que cualquier persona del equipo pueda seguir para levantar el proyecto.
- Un registro de tu uso de Agente de IA: qué le pediste, dónde te ayudó, dónde te equivocaste guiándolo, qué decidiste no delegarle.

3. Reglas del juego

Tiempo

- 3 días corridos desde que recibes este documento.

Uso de IA

- Claude Code, Codex u otra es parte de la prueba, no una trampa. Esperamos que lo uses.
- Lo que evaluamos es CÓMO lo usas: qué decides delegarle, cómo lo guías, dónde lo corriges, qué decisiones te reservas para ti.
- Lleva un log breve (puede ser un archivo AI_LOG.md) con los prompts más relevantes y una reflexión corta sobre cada uno.

Preguntas durante la prueba

- Puedes hacer las preguntas que consideres pertinentes a jrain@primuscapital.cl
- Si no preguntas nada y asumes algo, déjalo documentado como supuesto.

Entrega

- Repositorio Git (público o privado con acceso a nosotros).
- README claro.
- Un documento DECISIONS.md con las 5-7 decisiones más importantes que tomaste y por qué.
- AI_LOG.md con el registro de uso de Claude Code.

4. Criterios de aceptación mínimos

Funcionalidad

- El sistema levanta siguiendo las instrucciones del README en un equipo limpio.
- Existe al menos un flujo principal usable end-to-end: el usuario entra al frontend, interactúa, y los cambios se persisten en la base de datos.
- Los datos persisten entre reinicios de la aplicación.

Backend

- Backend en Go o NestJS (no otro stack salvo conversación previa).
- API expuesta vía HTTP con endpoints documentados (Swagger, README, o archivo aparte).
- Validación de inputs en los endpoints expuestos.

Base de datos

- Base de datos relacional (PostgreSQL)
- Diagrama del modelo de datos en el repo (dbdiagram.io, imagen, lo que sea legible)

Frontend

- Frontend en React que consume el backend (no datos mockeados en el cliente).
- Manejo visible de estados de carga y error (no pantallas en blanco mientras se hace fetch).
- Funcional en un navegador moderno (Chrome/Firefox actual).

Infraestructura local

- Levantar el sistema completo no debería tomar más de 10 minutos a alguien que clone el repo por primera vez.
- Variables de entorno gestionadas vía archivo (no valores hardcodeados en el código).
- Un .env.example versionado que muestre qué variables se necesitan.

Documentación

- README con: descripción breve, requisitos, cómo levantar, cómo probar el flujo principal.
- DECISIONS.md con las 5-7 decisiones más relevantes que tomaste.
- AI_LOG.md con el registro de uso de Claude Code.

Repositorio

- Historial de commits con mensajes descriptivos (no un único commit gigante al final).
- .gitignore apropiado al stack elegido.

Nota: lo anterior es lo mínimo. No esperes que cumplirlo sea suficiente para destacar. Lo que decidas hacer **además** de esto (y por qué) es parte central de lo que evaluamos.

5. Presentación final

Tras la entrega, agendaremos una sesión de 45 minutos:

10 min	Presentación tuya: el problema como lo entendiste, qué construiste, qué dejaste fuera y por qué.
10 min	Demo en vivo del sistema.
20 min	Preguntas, profundizaciones técnicas y discusión de trade-offs.
5 min	Cómo extenderías esto si tuvieras 2 semanas más.

6. Qué vamos a evaluar (y qué no)

Para que sepas dónde poner energía, te contamos qué pesa más al evaluar:

Pesa mucho

- Cómo descompusiste un requerimiento ambiguo en algo concreto y entregable.
- Que el sistema funcione end-to-end: frontend, backend y base de datos conectados, con un flujo principal usable.
- Comunicación: qué preguntaste, qué supuestos hiciste explícitos, cómo defiendes tus trade-offs.
- Implementación de la solución limpia, ordenada y con buenas prácticas.

Pesa, pero menos

- Belleza visual del frontend (que sea usable, no que sea Dribbble).
- Cobertura de tests (queremos los tests correctos, no muchos).
- Cantidad de features (preferimos 3 flujos sólidos a 5 flujos a medias).

No pesa

- Que uses Go vs NestJS (cualquiera está bien).
- Que el código sea "perfecto" según un estándar abstracto.
- Que tengas conocimiento previo del rubro.

6. Pistas honestas

No son obligatorias, pero te ahorrarán tiempo:

1. Decide tu alcance ANTES de programar. ¿Qué entra en el MVP de 3 días? ¿Qué queda para después? Defiende ese alcance.
2. Una decisión bien argumentada > una decisión "correcta" sin argumento. No buscamos la respuesta única; buscamos cómo razonas.
3. Si algo te parece raro o ambiguo en este enunciado, probablemente lo dejamos así a propósito.